FlightBookPlatform Review Pack
- Table of Contents

Requirement Responses
Requirement Responses
- Overview
- 1. Deploy the application online to ease the review process
- 2. Scale beyond sample data
- 3. Use data storage(s) provisioned within the environment
- 4. Implement automated software tests
- 5. Document Web Services
- 6. Allow flights to be restricted to a preferred airline
- 7. Sort trip listings
- 8. Paginate trip listings
- 9. Allow flights departing and/or arriving in the vicinity of requested locations
- 10. Support open-jaw trips, a pair of one-ways getting from A to B then from C to A
- 11. Support multi-city trips, one-ways (up to 5) from A to B, B to C, C to D, etc.

README
Flight Book Platform — Trip Builder
- Project Overview
  - Supported Trip Types
  - Business Rules
- Tech Stack
- Quick Start
  - Prerequisites
  - 1. Clone the repository
  - 2. Configure environment
  - 3. Build and start containers
  - 4. Open the application
- Source Code Architecture
- Default Credentials
- API Endpoints
  - Public
  - Authenticated (Sanctum)
- Artisan Commands
- Scheduled Jobs
- Sample Data
- Project Structure
- How to add a job to be queue
- Troubleshooting

Quick Start Guide
FlightBookPlatform — Local Setup Guide
- Software Versions
  - Infrastructure And Runtime Versions
  - Backend Package Versions
  - Frontend Package Versions
- Prerequisites
- Quick Start
- ** in the .env file replace AVIATIONSTACK_API_KEY=api_key with necessary key
- Service URLs
- Database Credentials (default)
- Custom Environment Variables
- Stopping the App

AWS Online Deployment
AWS Online Deployment
- Overview
- Production Deployment Target
- Deployment Workflow
- Environment Configuration
- Laravel Services Managed In Production
  - Web Application
  - Queue Processing
  - Database And Cache

Document Web Services
Document Web Services
- Overview
- 1. Public Web Services
  - Homepage Service
  - Public Trip Sharing Service
- 2. Authentication Services
  - Login Service
  - Logout Service
  - Registration Service
  - Password Recovery Service
  - Email Verification Service
  - Password Confirmation And Two-Factor Services
- 3. Authenticated User Web Services
  - User Dashboard Service
  - Help And Tutorial Services

- Profile Service
- Trips And Planning Pages

4. Client SPA API Services
   - User Trip Management API
   - Quick Trip Creation And Editing API
   - Flight Search And Builder Support API
5. Public REST API Services
   - Airline API
   - Airport API
   - Flight API
   - Authenticated Trip API
6. Administrative Web Services
   - Admin Dashboard Service
   - Admin User Management Service
   - Admin Airline Management Service
   - Admin Airport Management Service
   - Admin Flight Management Service
   - Admin Trip Management Service
   - Admin Flight Import Service
7. Utility Service
   - Sanctum CSRF Service
8. Summary

FlightBookPlatform Review Pack

Table of Contents

Requirement Responses

Requirement Responses

Overview

This document answers the requested project requirements . Each item below states the requirement and summarizes how it is implemented.

1. Deploy the application online to ease the review process

Status: Implemented

The application has a online deployment path through Bitbucket Pipelines, defined in bitbucket-pipelines.yml. The production deployment target is `https://quantumstackjobs.zentrianalytics.com/`, and the deployment process updates code on a remote Ubuntu host, rebuilds assets, runs migrations, reloads Nginx and PHP-FPM, and restarts Horizon.

Supporting deployment documentation is also available in aws_online_deployment.md.

2. Scale beyond sample data

Status: Implemented

The application is built on PostgreSQL and uses repository/service/controller layers rather than hardcoded sample-only logic. Listings are queried dynamically with filtering, pagination, sorting, and relational loading, which allows the application to operate beyond seeded demo records.

The application also includes a live data ingestion path through the AviationStack API, which is used to pull flight arrival information by airport IATA code and feed it into the system for processing. That import flow is implemented in AviationStackService.php, FetchFlightDataCommand.php, and ProcessAviationStackFlightsJob.php.

This broader scalability can be seen in the data-access and service layers such as FlightService.php, TripService.php, routes/web.php, and routes/api.php.

3. Use data storage(s) provisioned within the environment

Status: Implemented

The application uses PostgreSQL for primary relational data and Redis for cache, sessions, and queue workloads. These services are provisioned directly in the environment through docker-compose.yml for local/demo use and are also expected in production through deployment environment configuration.

The application also stores imported AviationStack JSON payloads on Laravel storage before processing them asynchronously, as shown in AviationStackService.php.

4. Implement automated software tests

Status: Implemented

The codebase includes both unit and feature tests. Service-layer unit tests exist under tests/Unit/Services, and controller/API feature tests exist under tests/Feature.

Examples include TripBuilderServiceTest.php, AdminTripControllerTest.php, and ClientTripBuilderControllerTest.php. The Bitbucket pipeline also defines lint and test steps in bitbucket-pipelines.yml.

5. Document Web Services

Status: Implemented

The application web services are documented in ApplicationEndpoints.md. That document groups the platform into public services, authentication services, authenticated user services, client SPA APIs, public REST APIs, admin services, and utility services.

This provides a presentation-ready summary of the available web service surface for interview or review purposes.

6. Allow flights to be restricted to a preferred airline

Status: Implemented

Preferred airline filtering is supported in the flight explorer and trip-builder flows. The client API accepts preferred airline IDs, and the flight search logic passes those restrictions into the search layer.

Relevant implementation points include ClientTripBuilderController.php, TripBuilderService.php, and FlightService.php.

7. Sort trip listings

Status: Implemented

Trip listing sort support exists in the application service and admin management layers. Admin trip listing endpoints explicitly accept sorting input, and trip data is returned through structured listing services instead of static unordered output.

The relevant flow is visible in AdminTripController.php and TripService.php.

8. Paginate trip listings

Status: Implemented

Trip listings are paginated both for users and for administrators. The application returns paginated trip collections through the service and repository layers, and those are exposed through the relevant API and admin endpoints.

This behavior is implemented through TripService.php, TripController.php, and AdminTripController.php.

9. Allow flights departing and/or arriving in the vicinity of requested locations

Status: Implemented

The trip builder supports nearby airport logic based on requested locations. It finds nearby airports around the selected departure and arrival airports and expands the search so results can include flights in the vicinity rather than only exact-airport matches.

This is implemented in TripBuilderService.php and depends on nearby-airport support exposed through AirportService.php.

10. Support open-jaw trips, a pair of one-ways getting from A to B then from C to A

Status: Implemented

Open-jaw trips are explicitly modeled and validated. The trip type exists in the enum, and the trip builder validates the required route shape so the second segment ends at the original departure airport while allowing a different departure city on the return leg.

This support is implemented in TripType.php and the leg validation rules in TripBuilderService.php.

11. Support multi-city trips, one-ways (up to 5) from A to B, B to C, C to D, etc.

Status: Implemented

Multi-city trips are explicitly supported up to five segments. The builder validates the leg count and enforces sequential route continuity so each leg departs from the previous leg's arrival airport.

This is implemented in TripType.php and TripBuilderService.php, where `MAX_MULTI_CITY_SEGMENTS` is set to `5`.

README

Flight Book Platform — Trip Builder

A fully functional Laravel flight booking web application and API for building and navigating trips for a single passenger.

Project Overview

This application allows users to search and book flights across airports, build multi-segment trips, and manage their bookings through a modern single-page application interface.

Supported Trip Types

Type	Description
One-Way	A flight getting from A to B
Round-Trip	A pair of one-ways: A → B then B → A
Open-Jaw	A pair of one-ways: A → B then C → A
Multi-City	Up to 5 one-ways: A → B → C → D → E

Business Rules

- A trip **must** depart no earlier than the time of creation
- A trip **must** depart no later than 365 days after creation
- Trip price = sum of all referenced flight prices
- Flights are priced per single passenger in a neutral currency

---

Tech Stack

Layer	Technology
Backend	PHP 8.4, Laravel 13

Layer	Technology
Frontend	Vue 3, Inertia.js v3, Vite
Auth	Laravel Breeze + Fortify
Queue	Laravel Horizon (Redis)
Database	PostgreSQL 17
Cache/Sessions	Redis 7
Web Server	Nginx
Mail (dev)	Mailpit
Containerisation	Docker Compose

Quick Start

Prerequisites

- Docker Desktop (Mac / Windows / Linux)
- Git

1. Clone the repository

```
git clone https://bitbucket.org/zentrianalytics/flightbookplatform.git
cd flightbookplatform
```

2. Configure environment

```
cp .env.example .env
```

Edit `.env` and set your AviationStack API key (free at aviationstack.com):

```
AVIATIONSTACK_API_KEY=your_key_here
```

3. Build and start containers

```
docker compose up --build
```

> First build takes a few minutes. Subsequent starts are fast.

The entrypoint script automatically: - Installs Composer dependencies - Installs npm dependencies and builds frontend assets - Waits for PostgreSQL to be ready - Runs database migrations - Seeds demo users, airlines, airports, and sample flights on first boot - Reprocesses bundled AviationStack JSON files on first boot

4. Open the application

Service	URL
Application	http://localhost
Mailpit (test mail)	http://localhost:8025
Horizon dashboard	http://localhost/horizon

If you want live AviationStack imports to work, set a valid `AVIATIONSTACK_API_KEY` in `.env` before starting the containers.

Source Code Architecture

The backend follows a repository pattern so business logic is separated from persistence logic. Controllers delegate to service classes such as FlightService.php, TripService.php, and FlightImportService.php, while data access is handled through repository interfaces and implementations under app/Repositories and app/Repositories/Contracts. This structure makes the code easier to test, maintain, and extend as the application grows beyond demo data.

Flight import and reprocessing workflows are handled asynchronously with Laravel Horizon and Redis-backed queues. Live AviationStack payloads are fetched by FetchFlightDataCommand.php, stored to JSON by AviationStackService.php, and processed in the background by ProcessAviationStackFlightsJob.php, allowing larger import workloads to be handled without blocking user-facing requests.

Default Credentials

Role	Email	Password
Admin	admin@flightbookplatform.com	password
User	user@flightbookplatform.com	password

API Endpoints

All API endpoints are prefixed with `/api/v1`.

Public

Method	Endpoint	Description
GET	`/api/v1/airlines`	List all airlines (paginated)
GET	`/api/v1/airlines/{airline}`	Show airline with flights
GET	`/api/v1/airports`	List all airports (paginated)
GET	`/api/v1/airports/{airport}`	Show airport with flights
GET	`/api/v1/flights`	List flights (filterable)
GET	`/api/v1/flights/{flight}`	Show flight details

Flight filters (query string)

Parameter	Example	Description
`departure`	`?departure=YUL`	Filter by departure airport IATA code
`arrival`	`?arrival=YVR`	Filter by arrival airport IATA code
`airline`	`?airline=AC`	Filter by airline IATA code

Authenticated (Sanctum)

Method	Endpoint	Description
GET	`/api/v1/trips`	List authenticated user's trips
POST	`/api/v1/trips`	Create a new trip
GET	`/api/v1/trips/{trip}`	Show trip with segments
PUT/PATCH	`/api/v1/trips/{trip}`	Update trip
DELETE	`/api/v1/trips/{trip}`	Delete trip

Artisan Commands

```
# Fetch live flight data from AviationStack (runs automatically every
        12 hours)
docker compose exec php php artisan flights:fetch

# Run database migrations
docker compose exec php php artisan migrate

# Seed sample data
docker compose exec php php artisan db:seed

# Monitor queues
docker compose exec php php artisan horizon
```

Scheduled Jobs

Schedule	Command	Description
Every 12 hours (06:00 & 18:00)	`flights:fetch`	Fetches live arrivals for 8 airports from AviationStack API, saves raw JSON to storage, and dispatches processing jobs

Schedule	Command	Description
		to the `flight-imports` Horizon queue

Sample Data

The database seeder includes the following sample data from the assignment specification:

Airlines: Air Canada (AC), WestJet (WJ), Porter Airlines (PD)

Airports: YUL (Montreal), YVR (Vancouver), YYZ (Toronto), YYC (Calgary)

Flights:

Flight	Route	Departure	Arrival	Price
AC301	YUL → YVR	07:35	10:05	$273.23
AC302	YVR → YUL	11:30	19:11	$220.63
AC101	YUL → YYZ	06:00	07:30	$149.00
AC201	YYZ → YVR	09:00	11:45	$310.00
WJ500	YYC → YYZ	08:15	12:30	$195.00
PD100	YYZ → YUL	14:00	15:45	$120.00

Project Structure

```
flightbookplatform/
├── app/
│   ├── Console/Commands/        # FetchFlightDataCommand
│   ├── Enums/                   # UserRole, TripType, TripStatus,
SegmentType
│   ├── Http/Controllers/Api/    # REST API controllers
│   ├── Jobs/                    # ProcessAviationStackFlightsJob
│   ├── Models/                  # Airline, Airport, Flight, Trip,
TripSegment, User
│   ├── Policies/                # TripPolicy (ownership
authorization)
│   └── Services/                # AviationStackService
├── database/
│   ├── migrations/
│   └── seeders/
├── docker-compose/
│   └── services/
│       ├── nginx/               # Nginx config
│       └── php-84/build/        # Dockerfile + entrypoint.sh
├── routes/
│   ├── api.php                  # Versioned API routes
│   └── web.php
├── docker-compose.yml
└── Documentation.md             # Docker troubleshooting guide
```

How to add a job to be queue

```
docker compose exec php php artisan tinker --execute '
App\Jobs\ProcessAviationStackFlightsJob::dispatch(
"aviation_stack/YUL/2026-04-04_18-42-03.json",
"YUL"
)->onQueue("flight-imports");
'
```

Troubleshooting

See Documentation.md for a detailed guide covering:

- PostgreSQL startup issues on macOS
- Docker named volume setup
- Redis and queue configuration
- Common error messages and fixes

Quick Start Guide

FlightBookPlatform — Local Setup Guide

FlightBookPlatform integrates with the AviationStack API to fetch live arrival flight data by airport IATA code. The application stores the raw JSON response, then processes it asynchronously through Laravel Horizon so imported airlines, airports, and flights can power trip planning, search, and admin management features.

Commands to process the data

sudo docker compose exec php php artisan flights:fetch sudo docker compose exec php php artisan flights:reprocess ## Stack

Service	Technology	Port
Web server	Nginx (alpine)	80
App	PHP 8.4 FPM	—
Queue	Laravel Horizon (Redis)	—
Scheduler	Laravel Scheduler	—
Database	PostgreSQL 17	5432
Cache/Queue	Redis 7	6379
Mail (test)	Mailpit	1025 / 8025

Software Versions

The application runs on a Docker-based Laravel and Vue stack with PostgreSQL, Redis, and Nginx. The versions below reflect the container images and the current locked application dependencies in this repository.

Infrastructure And Runtime Versions

Component	Version
PHP-FPM	`8.4`
Node.js	`22.x`
Nginx	`alpine` image
PostgreSQL	`17-alpine`
Redis	`7-alpine`
Mailpit	`latest` image

Backend Package Versions

Package	Version
Laravel Framework	`13.3.0`
Inertia Laravel	`3.0.1`
Laravel Fortify	`1.36.2`
Laravel Horizon	`5.45.5`
Laravel Sanctum	`4.3.1`
Laravel Telescope	`5.19.0`
PHPUnit	`12.5.16`

Frontend Package Versions

Package	Version
Vue	`3.5.32`
Inertia Vue 3	`3.0.2`
Vite	`8.0.6`
Tailwind CSS	`4.2.2`
Laravel Vite Plugin	`3.0.1`
`@vitejs/plugin-vue`	`6.0.5`

Prerequisites

- Docker Desktop installed and running
- Git

That's it. PHP, Composer, and Node are all handled inside the containers.

Quick Start

```
# 1. Clone the repo
git clone https://zentrianalytics-
        admin@bitbucket.org/zentrianalytics/flightbookplatform.git
cd flightbookplatform

# 2. Build and start all services
docker compose up --build


# 3. commands that should resolve most issues
docker compose up --build
sudo docker compose exec php npm install
sudo docker compose exec php npm run build
sudo docker compose exec php php artisan optimize
sudo docker compose exec php php artisan flights:fetch
sudo docker compose exec php php artisan migrate
```

On first boot the `php` container will automatically: - Copy `.env.example` → `.env` - Generate an `APP_KEY` - Install Composer dependencies - Install Node dependencies and build frontend assets - Wait for PostgreSQL to be ready - Run database migrations - Seed demo users and sample flight data - Reprocess bundled `aviation_stack` JSON files - Start PHP-FPM

If you want live flight imports from AviationStack to work, set a valid `AVIATIONSTACK_API_KEY` in `.env` before the first `docker compose up --build`.

The app will be available at: http://localhost

** in the .env file replace AVIATIONSTACK_API_KEY=api_key with necessary key

Service URLs

Service	URL
Application	http://localhost
Mailpit (UI)	http://localhost:8025

Database Credentials (default)

Use these to connect with DBeaver, TablePlus, or any SQL client:

Field	Value
Host	`localhost`
Port	`5432`
Database	`flightbookplatform`
Username	`flightbookplatform`
Password	`secret`

> To use custom credentials, set `DB_DATABASE`, `DB_USERNAME`, and `DB_PASSWORD` in a `.env` file in the project root **before** running `docker compose up` for the first time.

Custom Environment Variables

Docker Compose reads your local `.env` file for these overrides:

Variable	Default	Description
`DB_DATABASE`	`flightbookplatform`	Postgres database name
`DB_USERNAME`	`flightbookplatform`	Postgres username
`DB_PASSWORD`	`secret`	Postgres password
`APP_PORT`	`80`	Host port for the web app
`DB_PORT_FORWARD`	`5432`	Host port forwarded to Postgres
`REDIS_PORT_FORWARD`	`6379`	Host port forwarded to Redis
`MAIL_PORT_FORWARD`	`1025`	Host port forwarded to Mailpit SMTP
`MAILPIT_UI_PORT`	`8025`	Host port for Mailpit web UI

Stopping the App

```
# Stop containers (keeps data)
docker compose down

# Stop and wipe ALL data (full reset)
docker compose down -v

docker compose up --build
```

AWS Online Deployment

AWS Online Deployment

Overview

FlightBookPlatform is configured for online deployment through Bitbucket Pipelines. The production deployment target is the live application at `https://quantumstackjobs.zentrianalytics.com/`.

The CI/CD pipeline in bitbucket-pipelines.yml connects to a remote Linux server over SSH, updates the codebase, prepares the Laravel environment, builds assets, runs migrations, and restarts the required services.

Production Deployment Target

Item	Value
Production URL	`https://quantumstackjobs.zentrianalytics.com/`
Deployment trigger	Push to `main` branch
CI/CD platform	Bitbucket Pipelines
Remote deploy method	SSH remote execution
Web stack	Nginx + PHP-FPM
Queue manager	Laravel Horizon
Cache / queue backend	Redis
Database	PostgreSQL

Deployment Workflow

The production deployment is defined under the `main` branch pipeline. When changes are pushed to `main`, Bitbucket Pipelines runs a deployment step named `Deploy to production`.

The remote deployment script performs the following actions:

1. Connects to the remote Ubuntu server over SSH.

2. Clones the repository on first deploy if it is not already present.
3. Moves into `/var/www/flightbookplatform`.
4. Sets storage and cache directory permissions.
5. Checks out `main`, fetches the latest code, and hard-resets to the branch head.
6. Writes the production `.env` file from Bitbucket deployment variables.
7. Installs Composer dependencies without development packages.
8. Installs frontend dependencies and builds production assets.
9. Unzips the airports CSV asset bundle.
10. Runs Laravel database migrations in force mode.
11. Publishes Telescope assets.
12. Clears and rebuilds Laravel caches.
13. Reloads `php8.4-fpm` and `nginx`.
14. Creates or refreshes the `horizon.service` systemd unit.
15. Gracefully terminates old Horizon workers and restarts Horizon.

Environment Configuration

The pipeline writes the production environment file during deployment. The generated `.env` includes application, database, logging, mail, Redis, and AviationStack settings.

Important production variables referenced by the pipeline include:

- `APP_KEY`
- `APP_URL`
- `DB_HOST`
- `DB_DATABASE`
- `DB_USERNAME`
- `DB_PASSWORD`
- `SLACK_WEBHOOK_URL`
- `MAIL_MAILER`
- `MAIL_FROM_ADDRESS`
- `AVIATIONSTACK_API_KEY`

These values are stored securely in Bitbucket deployment variables rather than committed into the repository.

Laravel Services Managed In Production

Web Application

The application is served through Nginx with PHP 8.4 FPM. During deployment, both services are reloaded so the newest code and built assets are served immediately.

Queue Processing

Laravel Horizon is managed as a systemd service on the production server. The deployment script ensures the service definition exists, reloads systemd, gracefully terminates older workers, and restarts Horizon with the latest code.

Database And Cache

The application uses PostgreSQL for persistent data and Redis for cache, session, and queue workloads.

Document Web Services

Document Web Services

Overview

This document summarizes the web services exposed by FlightBookPlatform. The application provides a mix of public pages, authentication services, authenticated SPA endpoints, REST APIs, and admin management services.

The platform is built around flight discovery, trip building, booking management, and administration of core travel data such as users, airlines, airports, flights, and trips. The endpoint groups below are organized to support a clear interview walkthrough from public entry points through user workflows and finally into administrative services.

1. Public Web Services

Homepage Service

`GET /`
This is the public landing page for the application. It introduces the platform and gives guests access to sign in or create an account.

Public Trip Sharing Service

`GET /share/{token}`
This service displays a shared trip using a public token. It allows an itinerary to be viewed without requiring authentication.

2. Authentication Services

Login Service

`GET /login`
Displays the login form for existing users. It is the entry point into the authenticated application.

`POST /login`
Submits user credentials and creates an authenticated session. It is used by standard users and admins to access protected features.

Logout Service

`POST /logout`
Ends the current authenticated session. It returns the user to the public side of the application.

Registration Service

`GET /register`
Displays the account registration form. It is used by new users joining the platform.

`POST /register`
Creates a new user account. New registrations default to the Standard User role.

Password Recovery Service

`GET /forgot-password`
Displays the password reset request form. It allows a user to begin account recovery.

`POST /forgot-password`
Sends a password reset link to the user's email address. It supports users who cannot log in with their current password.

`GET /reset-password/{token}`
Displays the password reset form for a valid reset token. It is the second step of the password recovery workflow.

`POST /reset-password`
Stores the user's new password after reset validation passes. It completes the password recovery process.

Email Verification Service

`GET /email/verify`
Displays the email verification notice for authenticated users. It reminds users to verify their email before accessing restricted features.

`GET /email/verify/{id}/{hash}`
Processes the signed verification link. It marks the user's email as verified when the request is valid.

`POST /email/verification-notification`
Resends the email verification notification. It supports users who need a fresh verification link.

Password Confirmation And Two-Factor Services

`PUT /password`
Updates the password for the authenticated user. It is used inside account maintenance workflows.

`GET /user/confirm-password`, `POST /user/confirm-password`, `GET /user/confirmed-password-status`
These services support password confirmation before sensitive operations. They are part of the Fortify security workflow.

`GET /two-factor-challenge`, `POST /two-factor-challenge`
These services handle the two-factor authentication challenge during login. They add an additional verification step before full access is granted.

`POST /user/two-factor-authentication`, `DELETE /user/two-factor-authentication`, `POST /user/confirmed-two-factor-authentication`, `GET /user/two-factor-qr-code`, `GET /user/two-factor-secret-key`, `GET /user/two-factor-recovery-codes`, `POST /user/two-factor-recovery-codes`
These services manage two-factor authentication setup and recovery. They support enabling, confirming, viewing, regenerating, and disabling 2FA for the current user.

3. Authenticated User Web Services

User Dashboard Service

`GET /dashboard`
Displays the main application dashboard for authenticated and verified users. It serves as the main entry point after login.

Help And Tutorial Services

`GET /help`
Displays tutorial or onboarding help content. It supports users who have not yet completed the tutorial flow.

`POST /tutorial/seen`
Marks the tutorial as completed for the current user. This unlocks the rest of the gated application experience.

Profile Service

`GET /profile`
Displays the user profile page. It allows the user to review their personal information.

`PATCH /profile`
Updates the user's profile details. It supports changes such as name and email address.

`DELETE /profile`
Deletes the authenticated user's account. It completes account removal and ends the session.

Trips And Planning Pages

`GET /trips`
Displays the trips page with booking history and upcoming trips. It is focused on reviewing saved travel plans.

`GET /trip-builder`
Displays the trip builder interface. It supports building one-way, round-trip, open-jaw, and multi-city itineraries.

`GET /flights`
Displays the flight explorer page. It allows users to browse and filter flights before building or updating a trip.

`GET /trips/{trip}/map`
Displays a map view for a specific trip. It helps users visualize the itinerary and route sequence.

`GET /airports/map`
Displays airport information on a map. It supports geographic exploration of airport locations.

`GET /settings`
Displays the settings hub page. It acts as a central place for account and preference-related options.

4. Client SPA API Services

These services support the no-refresh single-page user experience. They are consumed directly by the Vue frontend components.

User Trip Management API

`GET /client-api/trips`
Returns the authenticated user's trip list. It supports the trips page and user account overview features.

`GET /client-api/trips/{trip}`
Returns a single trip with its related segments. It is used when a user views itinerary details.

`PATCH /client-api/trips/{trip}`
Updates trip details such as name, status, or departure information. It supports lightweight trip edits inside the SPA.

`DELETE /client-api/trips/{trip}`
Deletes a trip owned by the authenticated user. It supports self-service trip cleanup.

Quick Trip Creation And Editing API

`POST /client-api/trips/from-flight`
Creates a new trip directly from a selected flight. It reduces friction by allowing users to start a plan from the flight explorer.

`POST /client-api/trips/{trip}/segments/from-flight`
Appends a new segment to an existing trip. It supports growing a trip plan without restarting the booking flow.

`PUT /client-api/trips/{trip}/segments/replace-flight`
Replaces the selected flight in a one-way trip. It supports quick changes when the user wants a better option.

`POST /client-api/trips/{trip}/make-public`
Generates or returns a public sharing token for a trip. It enables public itinerary sharing through a shareable URL.

Flight Search And Builder Support API

`GET /client-api/flights`
Returns flight explorer results for the authenticated user. It supports filtering, sorting, and browsing flights in the SPA.

`GET /client-api/airports/search`
Returns airport suggestions based on user input. It supports search by airport name, airport code, and city code.

`GET /client-api/airports/map-data`
Returns airport data with coordinate information. It is used for map-based frontend features.

`GET /client-api/airlines/options`
Returns airline options for selectors and filters. It supports preferred airline filtering in the booking flow.

`POST /client-api/trip-builder/flights/search`
Searches flights for a trip-builder leg request. It supports route-based search, nearby airport expansion, filtering, sorting, and pagination.

`POST /client-api/trip-builder/book`
Books and stores a trip from the trip-builder payload. It validates the itinerary and persists the trip with its segments.

5. Public REST API Services

These services provide JSON endpoints for external consumers and decoupled clients.

Airline API

`GET /api/v1/airlines`
Returns a list of airlines. It supports external consumers that need airline reference data.

`GET /api/v1/airlines/{airline}`
Returns a single airline with related data. It is used when an integration needs airline detail information.

Airport API

`GET /api/v1/airports`
Returns a list of airports. It supports external consumers that need airport reference records.

`GET /api/v1/airports/{airport}`
Returns a single airport with related information. It is useful for integrations that need airport detail data.

Flight API

`GET /api/v1/flights`
Returns a filtered list of flights. It supports external browsing and flight search use cases.

`GET /api/v1/flights/{flight}`
Returns a single flight with its related records. It is used when a consumer needs complete flight details.

Authenticated Trip API

`GET /api/v1/trips`
Returns the authenticated user's trips through the Sanctum-protected API. It supports external clients that need trip listing functionality.

`POST /api/v1/trips`
Creates a new trip through the API. It supports programmatic or decoupled trip creation.

`GET /api/v1/trips/{trip}`
Returns a single trip owned by the authenticated user. It enforces ownership rules through policy authorization.

`PUT /api/v1/trips/{trip}` and `PATCH /api/v1/trips/{trip}`
Updates a trip through the API. It supports external editing workflows for trip details.

`DELETE /api/v1/trips/{trip}`
Deletes a trip through the API. It allows an authenticated client to remove a user-owned trip.

6. Administrative Web Services

These services are restricted to authenticated administrators. They power the admin SPA and the backend data-management workflows.

Admin Dashboard Service

`GET /admin`
Displays the admin dashboard page. It is the SPA shell for administrative tools.

`GET /admin/api/dashboard/stats`
Returns summary statistics for users, airlines, airports, flights, and trips. It powers dashboard widgets and high-level system reporting.

Admin User Management Service

`GET /admin/api/users`, `POST /admin/api/users`, `GET /admin/api/users/{user}`, `PATCH /admin/api/users/{user}`, `DELETE /admin/api/users/{user}`
These services provide full CRUD management for users. They support administration of accounts, roles, and account lifecycle operations.

Admin Airline Management Service

`GET /admin/api/airlines/options`, `GET /admin/api/airlines`, `POST /admin/api/airlines`, `GET /admin/api/airlines/{airline}`, `PATCH /admin/api/airlines/{airline}`, `DELETE /admin/api/airlines/{airline}`
These services manage airline records in the admin system. They support listing, lookup, option loading, creation, updating, and deletion.

Admin Airport Management Service

`GET /admin/api/airports/options`, `GET /admin/api/airports`, `POST /admin/api/airports`, `GET /admin/api/airports/{airport}`, `PATCH /admin/api/airports/{airport}`, `DELETE /admin/api/airports/{airport}`
These services manage airport master data. They support maintaining location, code, and mapping details for airports.

Admin Flight Management Service

`GET /admin/api/flights`, `POST /admin/api/flights`, `GET /admin/api/flights/{flight}`, `PATCH /admin/api/flights/{flight}`, `DELETE /admin/api/flights/{flight}`
These services manage flights within the admin application. They support search, inspection, creation, updates, and deletion of flight records.

Admin Trip Management Service

`GET /admin/api/trips`, `GET /admin/api/trips/{trip}`, `PATCH /admin/api/trips/{trip}`, `DELETE /admin/api/trips/{trip}`
These services let administrators inspect and manage trip records across all users. They support moderation, support, and data correction workflows.

Admin Flight Import Service

`POST /admin/api/flight-imports`
Uploads a raw JSON flight data file for processing. It stores the file and dispatches a background Horizon job to import the contained flight data.

7. Utility Service

Sanctum CSRF Service

`GET /sanctum/csrf-cookie`
Returns the CSRF cookie used by Sanctum and session-based SPA requests. It is typically called before authenticated browser-based API requests.

8. Summary

FlightBookPlatform exposes web services across three main layers: public access, authenticated user workflows, and administrator management services. The architecture supports both server-rendered entry pages and SPA-style JSON endpoints, which makes it suitable for interactive travel booking flows while still exposing reusable REST services for integrations and external clients.